

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE **COURSE CODE:** **CSA12**

COURSE **OUTCOME** COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

ANNEXURE A

Total Marks:25 100

Code of Conduct

APPLICATION BASED QUESTIONS

1P.Ahlen Abishek (Name / Reg No) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

3.
2.
1.

ANNEXURE A

A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of **data** are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

Question 1

4.

of such errors, and suggest
accurate and reliable operation.

A smart city
large volume
the system
bet

A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and overflow errors. Evaluate how segmentation (code, data, and stack segments) works in **Bo** analyze how improper segment handling can lead to such problems, and explain how instruction

A data communication system

represents packet information in hexadecimal format for ease of

execution and addressing modes must be managed to ensure correct program execution.

debugging and transmission, but a software engineer mistakenly interprets values, leading incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding	Application of	Concepts	Problem Solving Approach (20%)	Accuracy of Solution	Clarity (20%)
	of Problem					
Context (20%)			(20%)		(20%)	
Q1	3		5	3		2
						1

$N_{\square \square}$

└

Q2

3

4

4

7

2

1

6

Q3

4

4

3

1.5

14-

15

Q4

3

3

4

1.5

14.9

Q5

2

4

3

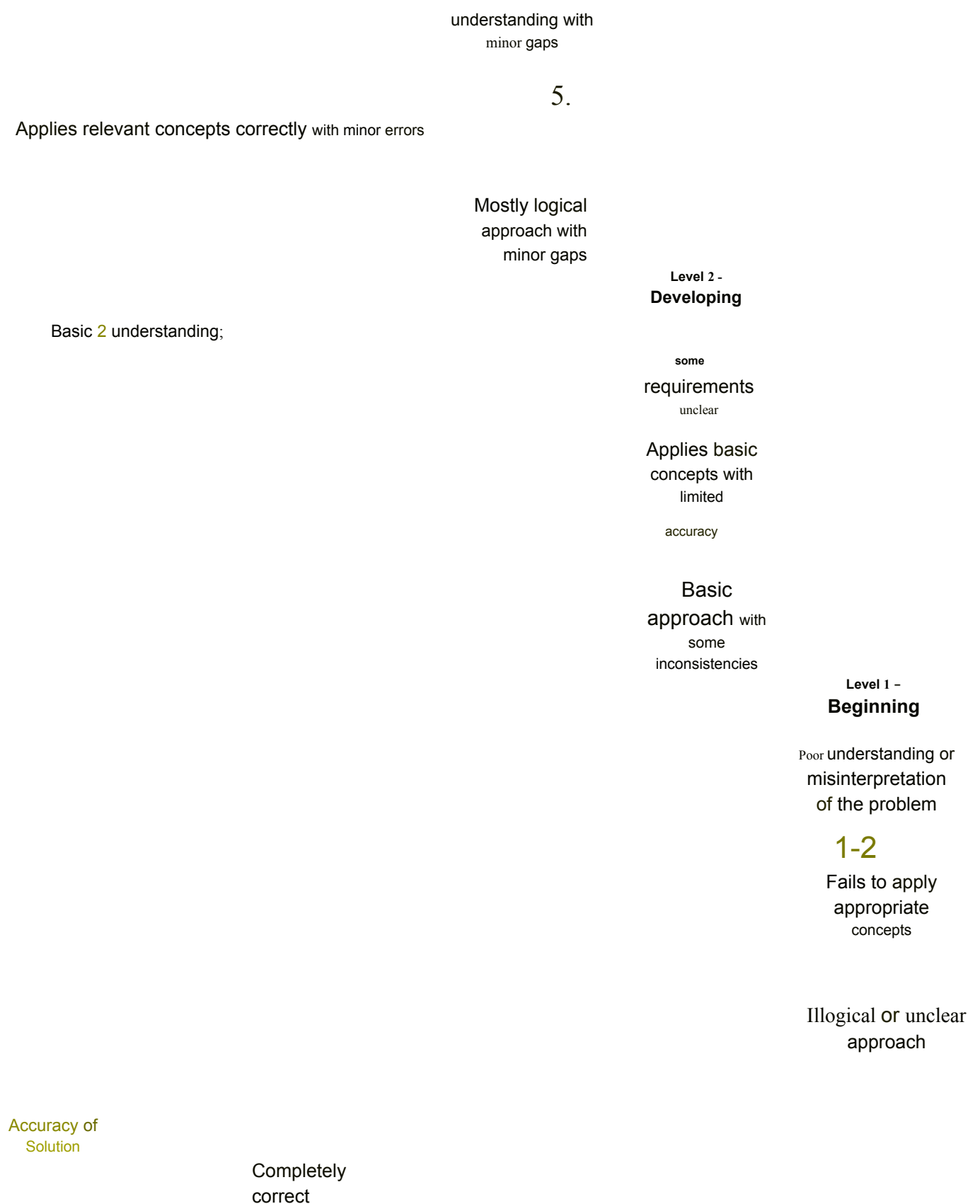
ce

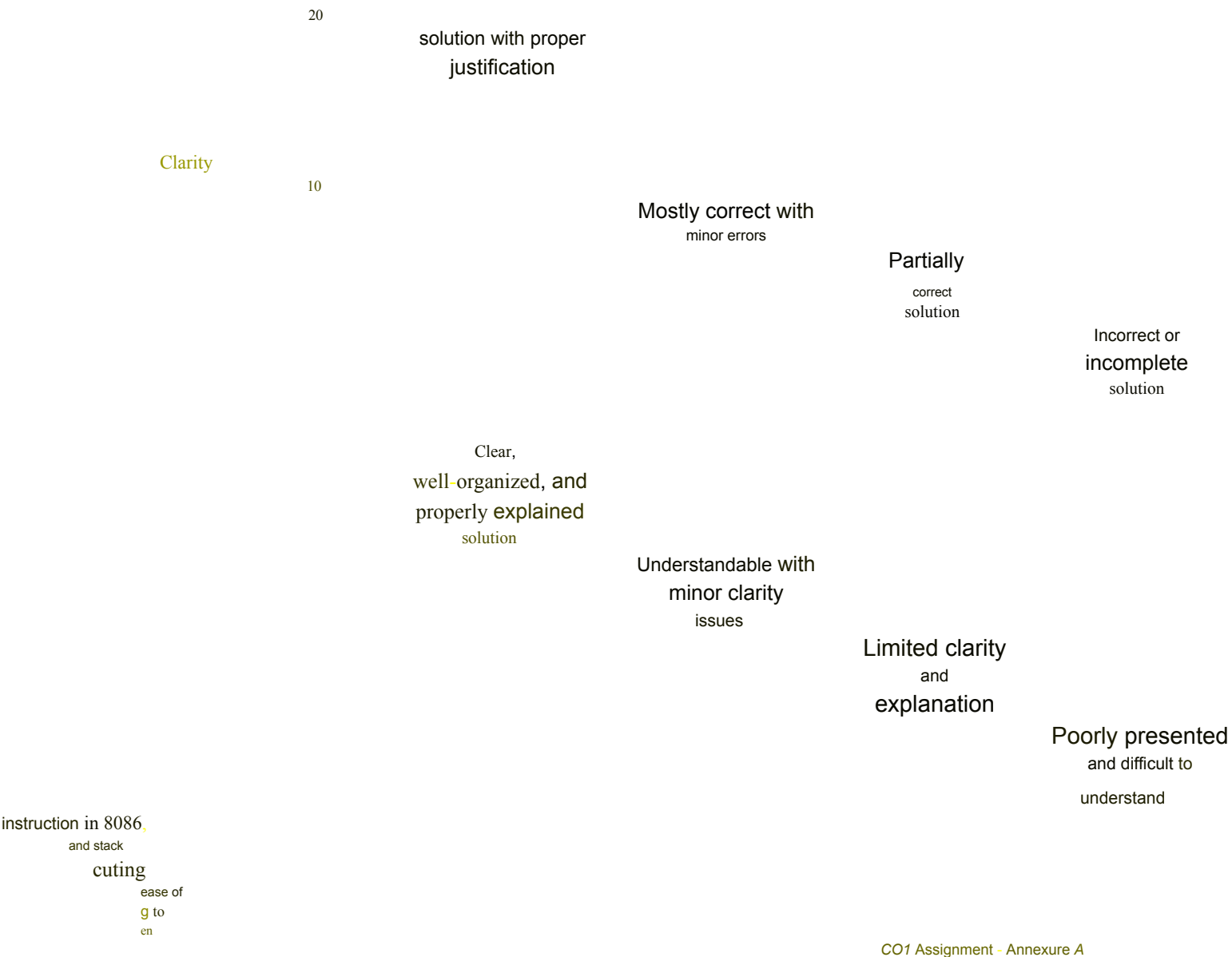
2

15

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3- Proficient
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies	
Application of Concepts	30		
Problem-Solving Approach	20	all requirements	
		Accurately applies appropriate concepts to solve complex problems	
		Logical, well-structured, and efficient approach	
			Good





Question 1

A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

1. Understanding the Problem Context

A smart city surveillance network continuously streams high-resolution video from numerous cameras. Each frame must travel from the camera (I/O device) into main memory before the CPU can fetch, decode and execute the instructions of the threat-detection algorithm on that data. During peak hours the volume and rate of incoming frames increases sharply, so the demand placed on the CPU- memory-I/O interaction rises well beyond normal load,

producing the observed lag and delayed alerts.

2. Analysis of CPU-Memory-I/O Interaction

Every incoming frame first triggers an I/O transfer into a memory buffer, then the CPU repeatedly executes fetch-decode-execute cycles to run detection routines, each cycle requiring memory reads (frame pixels, trained model parameters) and memory writes (intermediate results, alerts).

If the CPU, memory and I/O devices share one **common** communication path, only one data transfer can take place at any instant. When many camera controllers request bus access simultaneously with the CPU's own memory fetches, the requests queue up, causing bus contention, longer effective memory **access** time and CPU stalls while waiting for data.

The high interrupt rate generated by multiple cameras also forces the CPU to spend cycles on interrupt **servicing** and context switching instead of on detection computation, further reducing effective throughput and increasing detection latency during peak hours.

3. Role of Bus Structures

Single-bus architecture: a single shared bus carries address, data and control information among CPU, memory and every I/O device. It is simple and inexpensive, but because only one transfer can use the bus at a time, it becomes a severe bottleneck exactly in this scenario-camera data transfers and CPU memory accesses compete for the same path, and the effective bandwidth available to each unit falls as more cameras are added.

Multi-bus architecture: separates traffic onto **dedicated paths** for example a fast CPU-memory (system) bus and one or more separate I/O buses, often supported by DMA (Direct Memory **Access**) controllers that move camera frames directly into memory without CPU involvement. This allows the DMA transfer of new video frames to **proceed** on the I/O bus concurrently with the CPU fetching and executing detection instructions on the system bus, greatly reducing contention and latency compared with a single shared bus.

Single-Bus Architecture

CPU

Memory

I/O (Cameras)

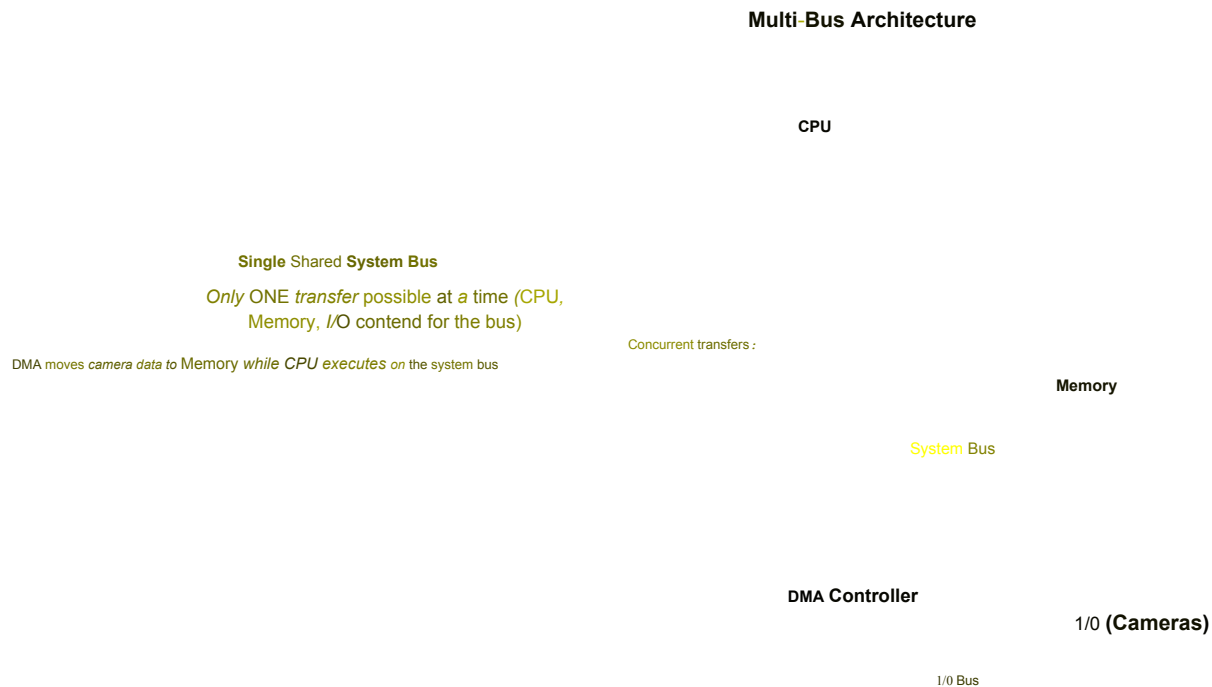


Fig. 1.

Single-bus (shared, contention-prone) vs Multi-bus with DMA (concurrent transfers).

4. Improvements in the Instruction Execution Cycle

Pipelining the **fetch-decode-execute** stages so several instructions are processed concurrently at different stages, raising instruction throughput without raising clock speed.

Cache memory placed between **CPU** and main memory to hold frequently used detection- algorithm code and data, reducing the number of slow **main-memory/bus** accesses.

DMA-based I/O so the CPU is not involved in copying every byte of video data, freeing **CPU** cycles purely for computation.

Prioritized, interrupt-driven I/O so that time-critical threat alerts are serviced ahead of routine frame handling.

Wider buses and burst-transfer modes so larger blocks of video data move per bus cycle, reducing the number of bus transactions needed.

5. Conclusion

The lag during peak hours arises from contention among the CPU, memory and I/O devices over a limited communication path while data volume is at its highest. Replacing a single shared bus with a multi-bus/DMA-based structure, together with pipelining, caching and prioritized interrupt handling in the instruction execution cycle, reduces access latency, increases concurrency, and restores real-time threat-detection performance.

Ques

Annexure A

CO1 Assignment - Annexure A

Question 2

A **real-time** stock trading application running on a processor executes millions of instructions **per second**, but users **report** delays in transaction processing during high load. The system **has** a high CPI **due** to frequent memory access and branch instructions. Examine how the **fetch-decode-execute cycle** impacts execution time in this scenario, and propose methods to **reduce** CPI and improve overall CPU performance using suitable architectural enhancements.

1. Understanding the Problem Context

The trading application executes an enormous number of instructions per second, yet a high Cycles-Per-Instruction (CPI) value — driven mainly by frequent memory accesses (reading live price data, order books) and frequent conditional branches (price/threshold checks that decide trade execution) — means that, on average, each instruction consumes more clock cycles than an ideal single-cycle instruction, so total execution time rises even though instruction count alone is high.

2. Performance Relationship

Execution Time = Instruction Count × CPI × Clock Cycle Time. Even with millions of instructions executed per second, a high average CPI multiplies directly into a longer execution time for the same instruction count, which is felt by users as transaction-processing delay.

3. Impact of the Fetch-Decode-Execute Cycle

Fetch: if the required instruction or operand is not already in cache (a cache miss), the fetch stage must wait for a slower main-memory access, adding stall cycles to that instruction.

Decode: for branch instructions, the next instruction address is not known with certainty until the branch condition is evaluated, creating a control hazard that can stall or misdirect the pipeline. Execute: load/store instructions that access memory for live price and order data may take multiple cycles if the data is not resident in cache, directly raising the CPI above the ideal value of 1. Because trading logic contains many memory-referencing instructions and many conditional branches, a large share of instructions incur these extra stall or flush cycles, so the measured CPI for the whole program is significantly higher than for a memory-light, branch-light program - this is the direct cause of the reported delay under high load.

Fetch-Decode-Execute Cycle

FETCH
(from memory / cache)

Pipelined Execution (overlap)

DECODE
(identify opcode,
operands)

Repeat for next
instruction

EXECUTE (ALU op,
memory access)

11

13

14

2

3

4

12

FI

D

E



A cache miss (Fetch stalls) or a branch misprediction (Decode/Execute hazard) breaks this overlap → extra stall/flush cycles → higher CPI

CO1 Assignment - Annexure A

4. Architectural Methods to Reduce CPI

Instruction pipelining, so fetch, decode and execute stages of different instructions overlap, moving the effective CPI of non-hazard instructions closer to 1.

Branch prediction (static or dynamic, e.g. branch history tables) to guess the likely branch outcome and pre-fetch along the predicted path, reducing the number of pipeline flushes caused by frequent conditional trade checks.

A cache hierarchy (L1/L2/L3) to reduce the average memory access time for the frequent price/order-book load and store instructions, cutting stall cycles.

Out-of-order / superscalar execution, allowing independent instructions to execute while another instruction waits on a memory operand, hiding memory latency.

Better data locality and register allocation in the trading code, reducing the number of

memory-access instructions needed per transaction.

5. Conclusion

The delay under high load is explained by an elevated average CPI, produced by memory-access stalls and branch hazards within the fetch-decode-execute cycle, which multiplies into longer execution time despite the very large raw instruction count. Pipelining, branch prediction, caching and out-of-order execution directly target these hazards, lowering CPI and restoring the low-latency response the trading application requires.

Question 3

COI Assignment - Annexure A

Question 3

An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

1. Understanding the Problem Context

In the 8085-based temperature controller, sensor values must be correctly fetched from memory or an input port, processed, and used to drive actuators. The problem states that incorrect readings are occasionally processed this points to an addressing-mode-level error, where the program is not always accessing the intended operand location.

2. Influence of 8085 Addressing Modes on Data Access

Immediate addressing (e.g. MVI A, 32H): the operand is a fixed constant written directly in the instruction. It never reads live data, so it is only correct for genuine constants such as thresholds, never for actual sensor readings.

- Register addressing (e.g. MOV A, B): the operand is inside a CPU register. This is fast, but is only correct if that register has already been loaded with the current, up-to-date sensor value before the instruction executes.

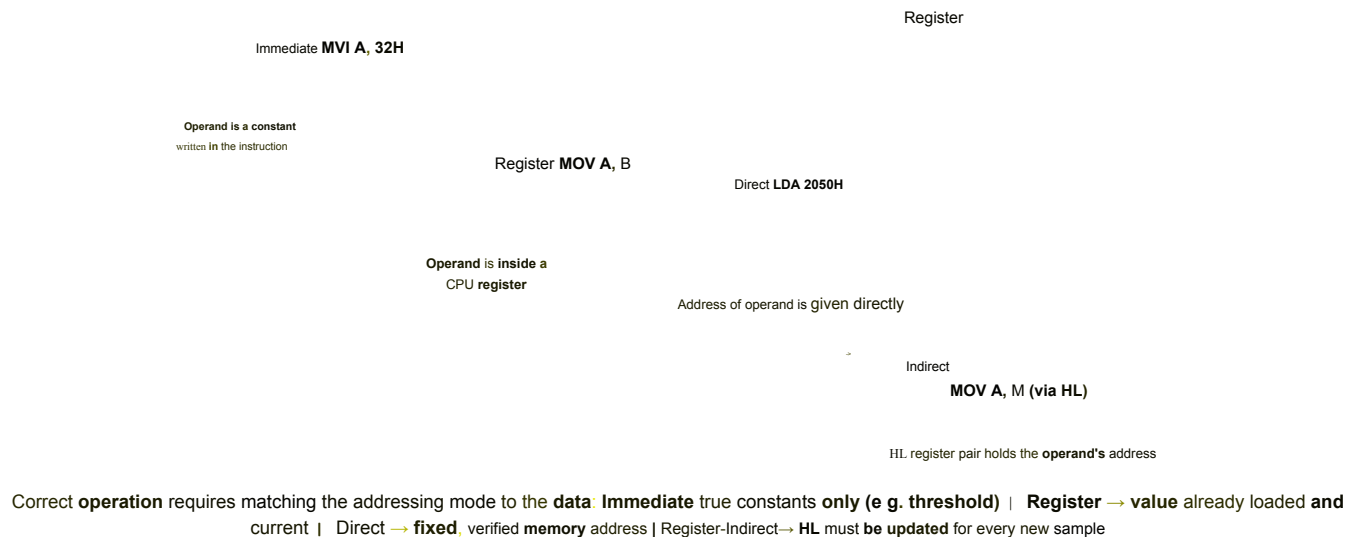
Direct addressing (e.g. LDA 2050H): the operand's memory address is given explicitly in the instruction - reliable for a single fixed location such as a sensor-data buffer, provided the address itself is typed correctly.

Register-indirect addressing (e.g. MOV A, M using HL): the address of the operand is held in a register pair; correctness depends entirely on HL being updated to point at the right memory location before each access- useful for reading a series of sensor samples one after

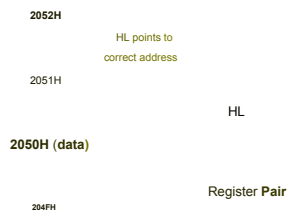
another.

- Implied/implicit addressing (e.g. CMA, RAL): the operand is implied by the **opcode** itself (usually the accumulator); **errors** here usually come from applying the operation to the wrong data already sitting in the accumulator.

8085 Addressing Modes - How the Operand is Located



Memory



COI Assignment - Annexure A

3. Possible Causes of the Errors

Using immediate addressing where indirect/direct addressing was required constant is processed instead of the live sensor reading.

a hard-coded

The HL pointer not being updated (incremented) between successive indirect reads, so the same or a wrong memory location is read repeatedly instead of the next sensor sample.

A typographical/off-by-one error in a direct address, causing the program to read an adjacent memory location that belongs to a different sensor or variable.

Misreading the data format at the addressed location (e.g. treating BCD sensor output as pure binary), which corrupts the interpreted value even though the correct address was accessed.

4. Using the 8085 Instruction Set for Accurate and Reliable Operation

Match the addressing mode to the nature of the data: register-indirect (HL-based) addressing with disciplined pointer increment for reading successive sensor samples; direct addressing only for single, fixed, well-verified memory locations; immediate addressing reserved strictly for true constants such as set-point thresholds.

Use IN/OUT instructions correctly with the exact port address for sensors/actuators connected as I/O-mapped devices.

Encapsulate repeated sensor-reading logic in subroutines (CALL/RET) with correct PUSH/POP use, so addressing is standardized and the stack is not corrupted.

Insert validity checks after each read using compare instructions (e.g. CPI) and conditional jump instructions (JZ, JC, JNZ) to detect out-of-range values before they are used to drive actuators.

Verify effective addresses computed by the program using an assembler/simulator during testing before field deployment.

5. Conclusion

Reliable operation of the temperature-control unit depends on choosing the addressing mode that matches the true nature of each operand constant, register-held, fixed memory location, or pointer-based sample and on disciplined pointer/stack management. Applying the 8085 instruction set with these practices, together with range-validation instructions, prevents the misreads that were causing incorrect temperature processing.

A

CO1 Assignment - Annexure A

Question 4

A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

1. Understanding the Problem Context

The 8086 uses segmented memory so that several programs can share the 1 MB physical address space generated from 16-bit registers. The reported symptoms - incorrect data access and stack overflow - both point to segment registers not being managed correctly across the multiple running programs.

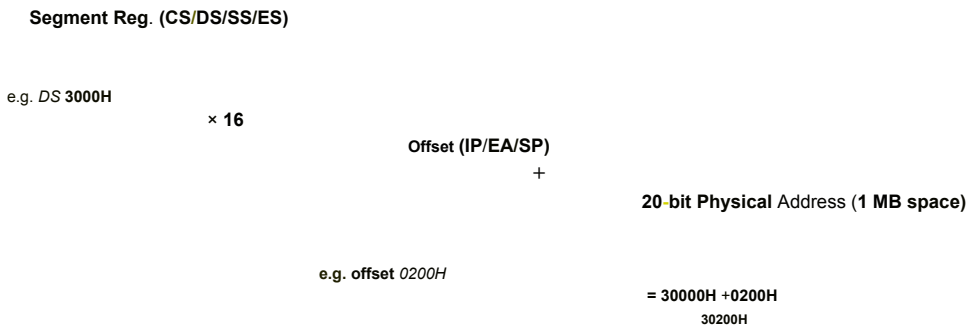
2. How 8086 Segmentation Works

The 8086 has four segment registers: CS (Code Segment), DS (Data Segment), SS (Stack Segment) and ES (Extra Segment). A physical address is formed as (Segment register value $\times$ 16) + Offset, where

the **offset** comes from IP for code, or from the effective address computed by the instruction's addressing mode (e.g. displacement, or BX/SI/DI based) for data and stack operands. This gives a 20- bit (1 MB) physical address from 16-bit segment and offset values.

CS:IP always points to the next instruction to be fetched. DS supplies the default segment for data operands unless overridden. SS:SP (with BP for stack-frame access) defines the stack area used for return addresses, saved registers and local variables, which grows downward from the top of the stack segment. ES is commonly used as an additional data segment, for example as the destination segment in string instructions such as MOVS and STOS.

8086 Segmented Memory Addressing



Each program needs CS, DS, SS reloaded to its **OWN segment** base before it runs. If DS is left pointing at another program's data segment → **wrong** data is read/written.

If **segments** overlap or **SS** is too small, SP can move **past the segment boundary** during PUSH / CALL → **STACK OVERFLOW**, corrupting adjacent code or data.

1 MB Address Space

ES - Extra

SS - Stack

DS - Data

CS - Code

Fig. 4- Physical address formation (Segment $\times$ 16+ Offset) and the four segments within the 1 MB space.

3. How Improper Segment Handling Causes the Reported Problems

CO1 Assignment - Annexure A

- Incorrect data access: if DS is not explicitly reloaded to point at the correct data segment for the program/module currently executing—for instance, a leftover DS value from a previously running program

every data instruction computes its physical address in the wrong segment, reading or writing memory belonging to another program.

Overlapping segments: because 8086 segment base addresses are only shifted by 4 bits, segments can be positioned so that they overlap in physical memory; if the code, data and stack segments of different programs are not allocated to non-overlapping regions, one program can silently overwrite another's memory.

- Stack overflow: SS:SP defines the stack; if the allocated stack segment is too small, or the program performs deep/uncontrolled recursion, or PUSH/POP and CALL/RET operations are not balanced, SP can move past the start of the stack segment and corrupt adjacent data or code memory — the classic stack-overflow symptom.
- Interrupt/task-switch handling that fails to save and restore all segment registers can also leave a wrong CS or SS in effect after control returns to a program, causing it to execute or address the wrong memory region.

4. Managing Instruction Execution and Addressing Modes for Correct Operation

- Explicitly reload DS, ES and SS at the start of each program or procedure using MOV instructions; never assume a segment register still holds the correct value after a task switch.
- Use segment-override prefixes explicitly whenever an instruction must access a segment other than its default, rather than relying on assumed defaults.
 - Allocate non-overlapping, adequately sized, paragraph-aligned segments for the code, data and stack of each program so that the available 1 MB space is properly partitioned among the multiple programs.
 - Size the stack segment generously and check SP against the segment's limit before deep calls or pushes; keep every CALL/RET and PUSH/POP pair balanced.
 - Use based/indexed addressing modes (e.g. [BX+SI+displacement]), or BP-based addressing for stack-frame variables) with a clear understanding of which segment register each mode defaults to.
 - On every interrupt or task switch, save all segment registers along with the general registers and

flags on the stack, and fully restore them before returning control.

5. Conclusion

8086 segmentation gives flexible access to a 1 MB address space from 16-bit registers, but this flexibility depends entirely on disciplined segment-register management. Unreloaded or overlapping segments and unmanaged stack growth are the direct causes of the incorrect data access and stack-overflow errors; explicit segment initialization, non-overlapping allocation, adequate stack sizing and balanced stack operations prevent them.

Question 5

COLAY

Annexure A

A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

1. Understanding the Problem Context

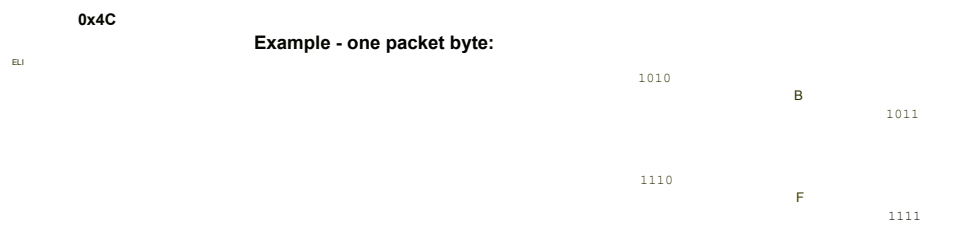
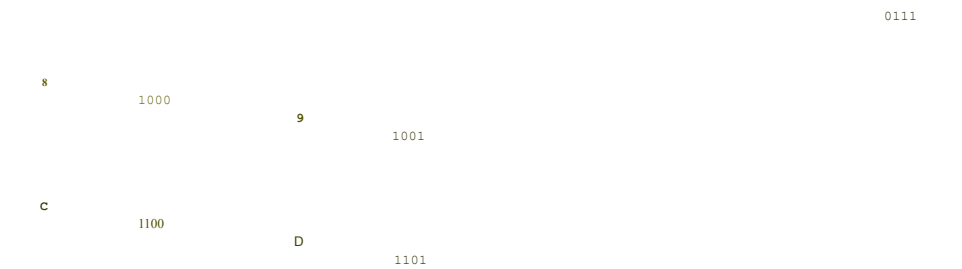
Digital systems operate internally in binary, but binary strings are long and error-prone for humans to read, write or debug - a single 32-bit value has 32 binary digits. Hexadecimal is used as a compact, human-readable shorthand for packet headers, memory dumps and addresses because each hexadecimal digit maps exactly to a 4-bit binary group (a nibble), giving a lossless one-to-one correspondence with the underlying binary data.

2. Importance of Hexadecimal-Binary Conversion

Because every bit in a packet field can carry real meaning - an opcode, an address, a length field, a flag- accurate conversion between hexadecimal and binary (and, where relevant, decimal, e.g. for IP address octets or port numbers) is essential; any conversion mistake changes the actual value the system will process, not merely its display.

Hexadecimal → Binary Conversion (1 Hex Digit= 1 Nibble = 4 Bits)

0	0000	1	0001	2	0010	3	0011
4	0100	5	0101	6	0110	7	



← correct nibble

split:

4 = 0100, C = 1100

A single **misread** digit **changes** the real value used by

the **system** wrong opcode, wrong address, or wrong packet field → wrong instruction executed **or** packet misrouted.

Checksum/CRC checks (using ALU status flags) and automated assemblers/analyzers catch such conversion errors.

Fig. 5 — Hex-to-binary nibble mapping and how one misread digit changes the transmitted byte.

3. How Conversion Errors Affect Instruction Execution and Data Processing

If a hexadecimal byte is misread or wrongly converted (for example transcribing 1A as 1C), the resulting numeric value used downstream - a packet length, an address, or an opcode is simply incorrect, leading to wrong memory locations being computed or the wrong opcode being interpreted.

In instruction execution specifically, machine code bytes are hexadecimal representations of binary opcodes and operands; a conversion mistake while hand-assembling or debugging code

Page 9 of 10

COI Assignment -
Annexure A

(common when working with 8085/8086 programs) causes the CPU to fetch and decode a different instruction than intended, which can produce unpredictable branches, wrong arithmetic/logic results, or a system crash.

- Because packet-type or header fields typically drive control flow (deciding which routine processes the packet), a single conversion error can misroute an entire processing pipeline - exactly the kind of systemic, real-time error described in the scenario.

4. Role of Efficient CPU Design and Benchmarking in Detecting/Minimizing Such Issues

Hardware-level checks: the ALU's status flags (carry, parity, zero, etc.) can be used by software to run automatic checksum, parity or CRC verification on received hexadecimal/binary data, catching many transcription **or** transmission errors before they reach instruction execution.

- Efficient CPU/memory design — error-correcting (ECC) memory and clearly defined number formats such as two's-complement representation and a fixed byte-order (endianness) reduces the ambiguity that leads to misinterpretation of values.
- Benchmarking with known test vectors (hexadecimal input sequences with known expected outputs) validates that conversion, instruction decoding and address computation are correct across the system; performance benchmarks such as CPI, execution time and branch- misprediction rate can also reveal unusual overhead that hints at an underlying data- interpretation bug, prompting further investigation.
- Using automated tools assemblers, disassemblers and protocol analyzers

instead of

manual hex/binary conversion reduces human error, and embedding validation routines (checksums, bounds checks) in software provides a further safety net.

5. Conclusion

Because binary is the machine's native language while hexadecimal is the human-readable shorthand for it, accurate conversion between the two is critical; a single conversion mistake can misdirect instruction execution or misinterpret packet data, with cascading effects in a real-time communication system. Careful CPU/memory design, automated conversion tools, error-checking hardware and systematic benchmarking together detect and minimize such errors before they affect system reliability